

# Nesne Yönelimli Programlama

Hafta 1: Dersin Amacı, OOP Vizyonu ve C++ Temelleri

Dr. Hüseyin Yüce

Marmara Üniversitesi Mühendislik Fakültesi

Mekatronik Mühendisliği Bölümü

# Ders İşleyişi: Devam ve Zamanında Gelme

Derse ve laboratuvara zamanında katılım, oturum düzeninin ve uygulamalı çalışmaların ön koşuludur.

- Teorik Dersler (2 Saat):
  - Kavramsal altyapı, bellek anatomisi, nesne yönelimli tasarım prensipleri ve mimari analizler.
- Uygulamalı Laboratuvar (2 Saat):
  - Kodlama pratikleri, canlı hata ayıklama (debugging), haftalık alıştırmalar ve vaka analizleri.
- Akademik Disiplin ve Zaman Yönetimi:
  - Ders başlangıcından sonra amfiye/laboratuvara giriş ders akışını ve odaklanmayı böler.
  - Laboratuvar çalışmalarında zamanında bulunmayan öğrenciler pratik çalışma puanlamasından mahrum kalır.

# Akademik Dürüstlük ve Kod Paylaşım İlkeleri

## Akademik Özgünlük İlkesi:

Tüm haftalık laboratuvar çalışmaları ve sınav çözümleri öğrencinin kendi bireysel zihinsel emeğinin ürünü olmalıdır.

- Birlikte Öğrenme vs. Kod Kopyalama:
  - Konuları ve algoritmaları arkadaşlarınızla tartışınız; ancak kod editörünüzdeki her satırı bizzat kendiniz yazınız.
- Yapay Zeka ve Çevrim İçi Kaynak Kullanımı:
  - Yapay zeka araçları kodun mantığını anlamak için bir öğrenme asistanı olarak kullanılabilir; ancak çözüm üretip kopyalamak öğrenme sürecini engeller.
- Özgünlük Denetimi:
  - Teslim edilen tüm kaynak kodlar intihal analiz yazılımlarıyla taranır; tespit edilen kopyalarda Marmara Üniversitesi disiplin yönergeleri uygulanır.

# Hafta 1: Hedefler ve OOP Vizyonu

Bu haftanın temel teorik hedefleri:

- Nesne Yönelimli Programlama (OOP) Vizyonu:
  - Prosedürel programlama © ile Nesne Yönelimli Programlama (C++) arasındaki kavramsal farkları kavramak.
  - Gerçek hayat mekatronik sistemlerinin durum (state) ve davranış (behavior) üzerinden modellenmesini öğrenmek.
  - Sınıf (Class) ile Nesne (Object) arasındaki mimari ayrımı netleştirmek.
- C++ Dili ve Mimarisi:
  - C++ dilinin tasarım felsefesini (*Zero-overhead Principle*) ve donanım kontrolündeki gücünü anlamak.
  - Önışlemci, derleyici ve bağlayıcıdan oluşan derleme mimarisini kavramak.

# Hafta 1: Uygulamalı C++ Hedefleri

Bu haftanın laboratuvar ve kodlama kazanımları:

- Geliştirme Ortamı ve Derleme:
  - Komut satırından `g++` ile C++ programı derleyip çalıştırabilmek.
  - Derleyici bayraklarının ( `-Wall` , `-Wextra` , `-std=c++14` ) kullanım amacını öğrenmek.
- Değişkenler ve Bellek Mimarisi:
  - C++ temel veri tiplerini, bellek boyutlarını ( `sizeof` ) ve değer aralıklarını kavramak.
  - Modern C++ süslü parantez `{}` ile liste ilklendirme (*Uniform Initialization*) standardını uygulamak.
- Girdi ve Çıktı Akışları:
  - `std::cout` , `std::cin` ve `std::cerr` akışlarını yönetmek.
  - `<iomanip>` kütüphanesi ile formatlı mühendislik telemetri tabloları basmak.
  - Tip dönüşümlerinde `static_cast` kullanımını benimsemek.

# 14 Haftalık Ders İzlenesi (1/3: C++ Temelleri ve Bellek)

| Hafta | Konu Başlığı                  | Odak Noktası ve Temel Kazanımlar                                                                                            |
|-------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1     | OOP Vizyonu ve C++ Temelleri  | Prosedürel vs OOP, derleme mimarisi, temel tipler, <code>&lt;iomanip&gt;</code> , <code>static_cast</code>                  |
| 2     | Kontrol Yapıları ve Döngüler  | Karar yapıları ( <code>if-else</code> , <code>switch-case</code> ), döngüler, Sonlu Durum Makinesi (FSM)                    |
| 3     | Fonksiyonlar ve Modülerlik    | Değer/referans/pointer aktarımı ( <code>const &amp;</code> ), aşırı yükleme, <code>.h</code> / <code>.cpp</code> modülerlik |
| 4     | Diziler ve Karakter Dizileri  | Çok boyutlu diziler, C-stringler, <code>std::string</code> , <code>std::vector</code> , arama/sıralama                      |
| 5     | Pointerlar ve Bellek Yönetimi | İşaretçi aritmetiği, <code>new[]</code> / <code>delete[]</code> , Stack vs Heap, bellek sızıntıları                         |

# 14 Haftalık Ders İzlencesi (2/3: Nesne Yönelimli Programlama)

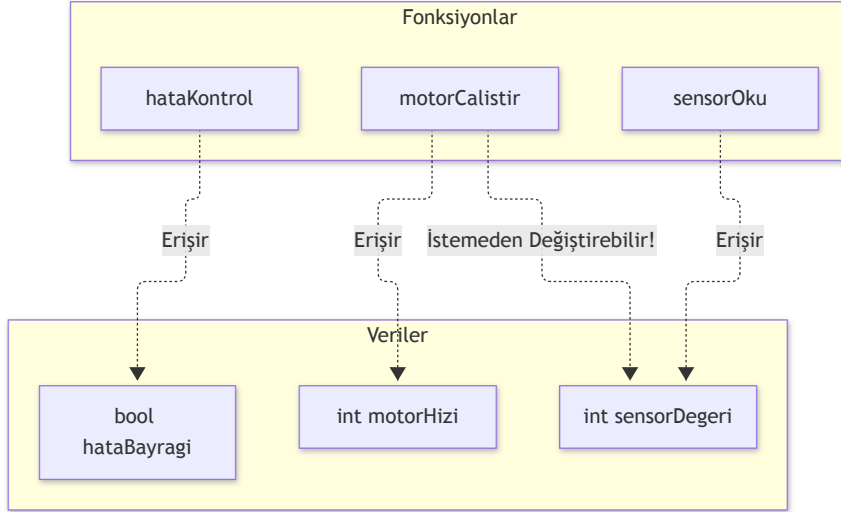
| Hafta | Konu Başlığı                         | Odak Noktası ve Temel Kazanımlar                                                                                          |
|-------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 6     | Yapılar, Birlikler ve Numaralandırma | <code>struct</code> , <code>union</code> , bit alanları, bayt hizalama/paketleme, <code>enum</code><br><code>class</code> |
| 7     | OOP'ye Giriş: Sınıflar ve Nesneler   | Sınıf/nesne, erişim belirleyicileri, yapıcılar, ilkendirme listesi, <code>friend</code>                                   |
| 8     | OOP: Kapsülleme ve Veri Gizleme      | Veri gizleme, Getter/Setter, soyutlama, <code>const</code> üye metotlar, <code>mutable</code>                             |
| 9     | Sınıfların İleri Düzeyi ve Kopyalama | <code>this</code> işaretçisi, statik üyeler, kopyalama kurucusu, Rule of Three                                            |
| 10    | Kalıtım ve Çok Biçimlilik            | Taban/türemiş sınıf, sanal fonksiyonlar ( <code>virtual</code> ), V-Table/V-Ptr, sanal yıkıcı                             |

# 14 Haftalık Ders İzlencesi (3/3: İleri OOP, Veri Yapıları ve Modern C++)

| Hafta | Konu Başlığı                           | Odak Noktası ve Temel Kazanımlar                                                                                                     |
|-------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 11    | Soyut Sınıflar, Arayüzler ve Dönüşüm   | Saf sanal metotlar, saf arayüz ( <code>interface</code> ), <code>override</code> , <code>dynamic_cast</code> , RTTI                  |
| 12    | Operatörlerin Aşırı Yüklenmesi         | Aritmetik, akış ( <code>&lt;&lt;</code> , <code>&gt;&gt;</code> ), indeks ( <code>[]</code> ), atama ( <code>=</code> ) operatörleri |
| 13    | Veri Yapıları ve Şablonlar (Templates) | Bağlı liste, yığın, kuyruk, şablon fonksiyonlar ve şablon sınıflar                                                                   |
| 14    | Modern C++, STL, RAII ve Dosya I/O     | STL konteyner ve algoritmaları, akıllı işaretçiler, taşıma semantiği, <code>fstream</code>                                           |

# Prosedürel Programlama © Mimarisi

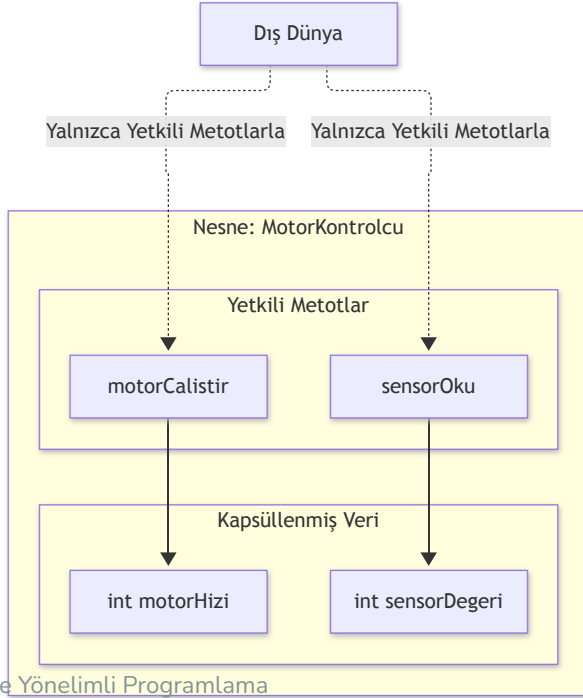
Fonksiyonlar ve Veriler birbirinden ayrılır:



- Veriler savunmasızdır; herhangi bir fonksiyon paylaşılan veya global veriyi kontrolsüzce bozabilir.
- Sistem büyüdükçe fonksiyon ve veri bağımlılıkları karmaşıklaşır (*Spaghetti Code*).

# Nesne Yönelimli Programlama (C++) Mimarisi

Veriler ve Fonksiyonlar nesne içinde paketlenir:



# Nesne Yönelimli Programlamanın Dört Temel Taşı

Tüm modern OOP mimarisi 4 sütun üzerine inşa edilmiştir:

## 1. Kapsülleme (Encapsulation):

- Veri ve bu veriyi işleyen metotların bir arada paketlenmesi, iç durumun dış müdahalelerden gizlenmesi (*Data Hiding*).

## 2. Soyutlama (Abstraction):

- Karmaşık arka plan uygulama detaylarının gizlenip, kullanıcıya yalnızca sade ve gerekli arayüzün sunulması.

## 3. Kalıtım / Miras (Inheritance):

- Var olan bir sınıftan yeni sınıflar türeterek kodun yeniden kullanılması (*Code Reuse*) ve hiyerarşi oluşturulması.

## 4. Çok Biçimlilik (Polymorphism):

- Aynı fonksiyon çağrısının, nesnenin çalışma anındaki gerçek türüne bağlı olarak farklı davranışlar sergileyebilmesi.

# Mekatronik Modelleme Örneği: Robot Kolu

- Nesne: Robot Kolu
- Durum (Özellikler / State):
  - Eklem açıları, anlık tork, çalışma sıcaklığı, motor hızı
- Davranış (Yetenekler / Methods):
  - `hareketEt()` , `tutucuKapat()` , `acilDurdur()`

Modelleme Notu: Robot kolunun anlık fiziksel büyüklükleri üye değişkenlerde, kinematik hareket kabiliyeti ise üye metotlarda temsil edilir.

# Mekatronik Modelleme Örneği: Sıcaklık Sensörü

- Nesne: Sıcaklık Sensörü (LM35 / PT100)
- Durum (Özellikler / State):
  - Pin numarası, kalibrasyon katsayısı, son okunan gerilim değeri
- Davranış (Yetenekler / Methods):
  - `sicaklikOku()` , `kalibreEt()` , `esikKontrol()`

Sensör Notu: Ham ADC okuması sınıf içinde kapsülendir; dış modüllere doğrudan filtrelenmiş ve kalibre edilmiş sıcaklık verisi sunulur.

# Mekatronik Modelleme Örneği: Otonom Araç

- Nesne: Otonom Araç
- Durum (State): Batarya seviyesi, direksiyon açısı, lidar engel mesafesi, GPS konumu
- Davranış (Behavior): `rotaHesapla()` , `seriTakipEt()` , `frenYap()` , `telemetryGonder()`

Temel Tasarım Kuralı: Bir nesne, kendi verisini (state) ve bu veriyi işleyen fonksiyonlarını (behavior) tek bir çatı altında birleştirir.

- Dış dünya nesnenin batarya seviyesini rastgele değiştiremez; yalnızca `sarjEt()` metoduyla yönetebilir.

# Sınıf (Class) Kavramı: Taslak ve Şablon

- Bir nesnenin taslağıdır (blueprint).
- Mimari çizim ve tip kalıbıdır; nesne oluşturulana dek bellekte yer kaplamaz (statik üyeler hariç).
- Nesnenin hangi verileri barındıracağını ve hangi davranışlara sahip olacağını tanımlar.

```
1 // Sınıf Tanımı (Taslak / Şablon)
2 class Motor {
3     public:
4         int rpm;
5         void calistir();
6 };
```

# Nesne (Object) Kavramı: Bellekteki Somut Varlık

- Sınıftan türetilen somut varlıktır (instance).
- Taslağa göre inşa edilmiş gerçek yapıdır; bellekte (RAM) yer tahsis edilir.
- Her nesnenin kendi bağımsız veri kopyası bulunur.

```
1  int main() {  
2      // Nesne üretimi (Somutlama - Instantiation)  
3      Motor m1; // RAM'de m1 için yer tahsis edildi  
4      Motor m2; // Bağımsız ikinci motor nesnesi  
5  
6      m1.rpm = 1500;  
7      m2.rpm = 3000;  
8      return 0;  
9  }
```

# Mekatronik Örneği: Motor Sınıfı ve Nesne Ayrımı

Bir robotik sistemde birden fazla motor bağımsız kontrol edilecekse:

```
1  class Motor {  
2  public:  
3      int pwmPini;  
4      int maxHiz;  
5      int anlikHiz;  
6  };
```

- Tek bir `Motor` sınıf şablonu tanımlanır.
- Bu şablondan birbirinden bağımsız çalışan nesneler üretilir:
  - `Motor solTeker;`
  - `Motor sagTeker;`
  - `Motor tutucu;`

# Motor Davranışları: Üye Fonksiyonlar (Metotlar)

Her motor nesnesi ortak üye fonksiyonları kendi üye değişkenleriyle yürütür:

```
1  class Motor {  
2  public:  
3      int anlikHiz;  
4  
5      void hizAyarla(int yeniHiz) {  
6          anlikHiz = yeniHiz;  
7      }  
8  
9      void durdur() {  
10         anlikHiz = 0;  
11     }  
12 };
```

- `solTeker.hizAyarla(100);` → Yalnızca sol motor nesnesinin anlık hızı güncellenir.
- `tutucu.durdur();` → Yalnızca tutucu motor nesnesinin hızı sıfırlanır.

# C++ Dili: Tarihçe ve Standartlar

- Bjarne Stroustrup (1979 - Bell Labs):
  - C dilinin donanıma doğrudan erişim hızını, Simula dilinin nesne yönelimli mimarisiyle birleştirdi ("*C with Classes*").
  - 1983 yılında resmi olarak C++ adını aldı ( `++` artırma operatörü).
- C++ Evrimi ve Standartları:
  - C++98 / C++03: İlk ISO standardı, Şablonlar (Templates), Standart Şablon Kütüphanesi (STL).
  - C++11 (Modern C++ Devrimi): `auto` , akıllı işaretçiler ( `smart pointers` ), taşıma semantiği ( `move` ), lambdalar.
  - C++14 / C++17: Genişletilmiş `constexpr` , `std::optional` , `std::variant` , dosya sistemi ( `<filesystem>` ).
  - C++20 / C++23: Modüller ( `modules` ), kavramlar ( `concepts` ), coroutines, ranges kütüphanesi.

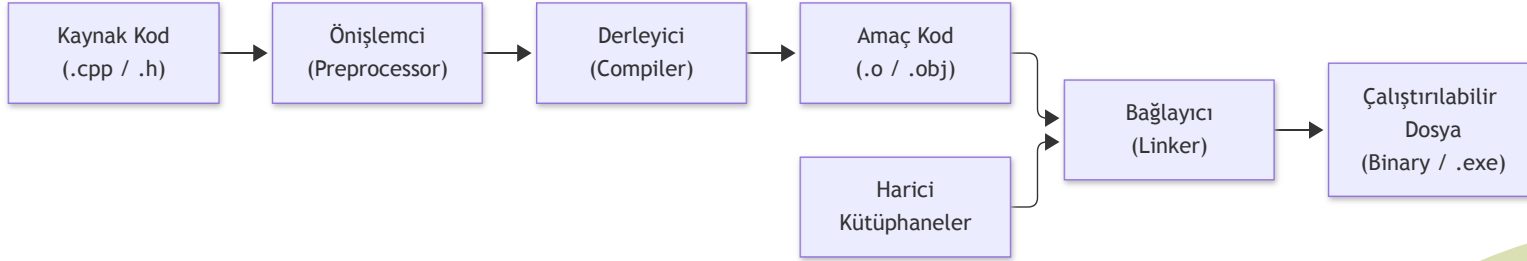
# Neden C++? Mekatronik ve Gömülü Sistemlerdeki Önemi

Mekatronik mühendisliğinde C++ neden vazgeçilmezdir?

- Sıfır Ek Yük Prensibi (Zero-overhead Principle):
  - *"Kullanmadığınız şeyin bedelini ödemezsiniz. Kullandığınız şeyi ise elinizle yazsanız daha verimli yazamazsınız."* (Bjarne Stroustrup)
- Doğrudan Donanım Erişimi:
  - İşaretçiler ve bellek adresleriyle mikrodenetleyici register'larına doğrudan yazabilme.
- Belirlenimci Bellek Yönetimi (Deterministic RAI):
  - Çöp toplayıcı (*Garbage Collector*) gecikmesi yoktur; nesne kapsamdan çıktığı mikro-saniyede yıkıcısı çalışır ve bellek temizlenir.
- Endüstri Standardı:
  - ROS / ROS 2 (Robot Operating System), OpenCV, MATLAB/Simulink kod üretimi ve otonom araç yazılımları C++ ile geliştirilir.

# C++ Derleme Mimarisi (Aşamalar)

C++ doğrudan çalıştırılan bir dil değildir; kaynak kod makine diline derlenir:



1. Önişlemci (Preprocessor): `#include` , `#define` , `#ifdef` yönergelerini metin seviyesinde çözer.
2. Derleyici (Compiler): C++ sözdizimini denetler, optimizasyon yapar ve amaç kod ( `.o` ) üretir.
3. Bağlayıcı (Linker): Fonksiyon çağrılarını, sınıf tanımlarını ve kütüphaneleri nihai ikili dosyada birleştirir.

# C++ Derleme Aşamaları: Karşılaşılan Hata Türleri

Hangi hata hangi aşamada tespit edilir?

- **Önişlemci Hataları:**
  - Bulunamayan başlık dosyaları ( `fatal error: iostream: No such file` ).
  - Hatalı makro tanımlamaları.
- **Derleme Zamanı Hataları (Compiler Errors):**
  - Sözdizimi (syntax) hataları (eksik noktalı virgül, kapatılmamış parantez).
  - Tür uyumsuzlukları ve geçersiz tür dönüşümleri.
  - Tanımlanmamış değişken veya fonksiyon kullanımı.
- **Bağlayıcı Hataları (Linker Errors):**
  - Gövdesi yazılmamış fonksiyon çağrılar ( `undefined reference to 'hesapla()'` ).
  - Birden fazla dosyada aynı değişkenin tanımlanması ( `multiple definition of 'x'` ).

# Komut Satırından Derleme (Tek Dosya)

C++ kaynak dosyasını doğrudan terminalden derlemek:

```
1  # Temel derleme
2  g++ main.cpp -o program
3
4  # Tüm uyarıları açarak derleme (Önerilen Akademik Standart)
5  g++ -Wall -Wextra -std=c++14 main.cpp -o program
6
7  # Programı çalıştırma
8  ./program
```

- `-Wall` : Tüm temel derleyici uyarılarını (*All Warnings*) etkinleştirir.
- `-Wextra` : Ekstra titiz uyarı kontrollerini devreye alır.
- `-std=c++14` : Kullanılacak C++ dil standardını belirler.

# Çoklu Kaynak Dosyası Derleme ve Bağlayıcı (Linker)

İlerleyen haftalarda sınıflarımızı başlık ( `.h` ) ve kaynak ( `.cpp` ) dosyalarına ayıracağız:

```
1 # İki kaynak dosyasını birlikte derleyip bağlama:
2 g++ Motor.cpp main.cpp -o robotProgrami
3
4 # Ayrı ayrı amaç koda dönüştürüp bağlama:
5 g++ -c Motor.cpp -o Motor.o
6 g++ -c main.cpp -o main.o
7 g++ Motor.o main.o -o robotProgrami
```

- `-c` : Yalnızca derle, bağlama yapma ( `.o` amaç kod üret).
- Büyük projelerde yalnızca değişen kaynak dosya derlenir; bağlayıcı (linker) nihai çalıştırılabilir binary'yi üretir.

# Geliştirme Ortamı Kurulumu (IDE ve Editörler)

Bu derste önerilen ve desteklenen geliştirme araçları:

- IDE / Editör Seçenekleri:
  - VS Code: Hafif, modüler, C/C++ uzantısı (Microsoft) ile güçlü entegrasyon. *(Önerilen)*
  - CLion: JetBrains tarafından geliştirilen tam donanımlı C++ IDE'si (öğrenci lisansı ile ücretsiz).
  - Visual Studio (Windows): Windows ortamında MSVC derleyicisi ile eksiksiz araç seti.
- Gerekli Eklentiler (VS Code):
  - C/C++ (Microsoft)
  - Code Runner veya CMake Tools

# Derleyici Kurulumu ve Doğrulama

Terminalden `g++ --version` komutu ile derleyici varlığını doğrulayınız:

- Linux (Ubuntu/Debian):

```
1 sudo apt update
2 sudo apt install build-essential gdb
```

- macOS:

```
1 xcode-select --install
2 # veya Homebrew ile:
3 brew install gcc
```

- Windows:

- MSYS2 / MinGW-w64: GCC/G++ araç setini kurar.
- veya Visual Studio Community: "Desktop development with C++" seçeneği.

# İlk Program: Merhaba Dünya!

```
1  #include <iostream>
2
3  int main() {
4      // Ekrana standart çıktı yazdır
5      std::cout << "Merhaba, Marmara Mekatronik!" << std::endl;
6      return 0;
7  }
```

Derleme ve Çalıştırma:

```
1  g++ -std=c++14 -Wall merhaba.cpp -o merhaba
2  ./merhaba
```

Konsol Çıktısı:

```
1  Merhaba, Marmara Mekatronik!
```

# Program Yapısının Satır Satır Analizi

- `#include <iostream>` :
  - Giriş/çıkış akış kütüphanesini (*Input/Output Stream*) programa dahil eder.
- `int main()` :
  - Programın ana giriş fonksiyonudur; işletim sistemi yürütmeye bu noktadan başlar.
- `std::cout` :
  - Standart karakter çıkışı nesnesi (*Character Output*). Ekrana veri akışı sağlar.
- `<<` (Akış Ekleme Operatörü):
  - Sağındaki veriyi solundaki çıktı akışına yönlendirir.
- `std::endl` :
  - Satır sonu ekler ( `\n` ) ve çıktı arabelleğini anında boşaltır (*flush*).
- `return 0;` :
  - İşletim sistemine programın başarıyla tamamlandığını (çıkış kodu 0) bildirir.

# Derleyici Hata Mesajlarının Analizi

Sözdizim hatası içeren kod örneği:

```
1  #include <iostream>
2  int main() {
3      std::cout << "Test" // Noktalı virgül EKSİK!
4      return 0;
5  }
```

Derleyici Çıktısı:

```
1  main.cpp: In function 'int main()':
2  main.cpp:4:5: error: expected ';' before 'return'
3      4 |     return 0;
4      |     ^~~~~~
```

# Hata Mesajı Okuma Stratejisi

- `main.cpp` : Hatanın tespit edildiği kaynak dosya adı.
- `4:5` : Satır 4, Sütun 5 (Derleyici hatayı noktalı virgülün eksikliğini fark ettiği bir sonraki satırın başında yakalar).
- `error` : Derleme sürecini durduran kritik hata; çalıştırılabilir dosya üretilmez.
- `expected ';' before 'return'` : Hatanın açık nedeni: `return` ifadesinden önce `;` karakteri bekleniyordu.

**Uygulama Notu:** Derleyici hata listesini incelerken her zaman en baştaki ilk hatadan başlayınız. İlk satırdaki tek bir noktalı virgül eksikliği derleyicinin durumunu bozarak onlarca sahte ardışık hataya yol açabilir.

# Yorum Satırları ve Kod Dokümantasyonu

```
1 // Tek satırlık açıklama
2
3 /*
4  * Çok satırlı
5  * açıklama bloğu
6  */
7
8 /// Doxygen formatında dokümantasyon
9 /// @brief Motorun PWM değerini ayarlar
10 /// @param pwm 0-255 arası hız değeri
11 void pwmAyarla(int pwm);
```

- Temiz kod, kendi kendini anlatan koddur; anlamlı değişken ve fonksiyon adları seçilmelidir.
- Yorum satırlarında yalnızca "ne yapıldığı" değil, "tercih edilen mimarinin gerekçesi" belirtilmelidir.

# C++ Temel Veri Tipleri (Temel Türler)

Bellekte değişken tanımlanırken veri tipinin açıkça bildirilmesi zorunludur (*Statically Typed*):

| Tip           | Tip Adı                | Boyut (Tipik)   | Değer Aralığı                                                                  |
|---------------|------------------------|-----------------|--------------------------------------------------------------------------------|
| Tamsayı       | <code>int</code>       | 4 Bayt (32 bit) | $-2.1 \times 10^9 \dots + 2.1 \times 10^9$                                     |
| Kısa Tamsayı  | <code>short</code>     | 2 Bayt (16 bit) | $-32,768 \dots + 32,767$                                                       |
| Uzun Tamsayı  | <code>long long</code> | 8 Bayt (64 bit) | $\approx \pm 9 \times 10^{18}$                                                 |
| Tek Duyarlık  | <code>float</code>     | 4 Bayt (32 bit) | $\approx 7$ basamak ondalık hassasiyet                                         |
| Çift Duyarlık | <code>double</code>    | 8 Bayt (64 bit) | $\approx 15 - 17$ basamak ondalık hassasiyet                                   |
| Karakter      | <code>char</code>      | 1 Bayt (8 bit)  | ASCII karakter kümesi: <code>'A'</code> , <code>'0'</code> , <code>'\n'</code> |
| Mantıksal     | <code>bool</code>      | 1 Bayt (8 bit)  | <code>true</code> (1) veya <code>false</code> (0)                              |

# Değişken Tanımlama Kuralları

C++ dilinde değişken tanımlarken uyulması gereken temel kurallar:

- Geçerli İsimlendirme Kuralları:
  - Harf ( `a-z` , `A-Z` ) veya alt çizgi ( `_` ) ile başlamalıdır.
  - Rakamla başlayamaz; ancak sonraki karakterlerde rakam bulunabilir ( `sensor1` , `motor_2` ).
  - Boşluk ve özel karakterler ( `-` , `+` , `@` , `#` , `$` ) içeremez.
  - Büyük/küçük harfe duyarlıdır (*Case-Sensitive*): `hiz` , `Hiz` ve `HIZ` üç farklı değişkendir.
- C++ İsimlendirme Teamülleri:
  - Değişken ve fonksiyonlar için: `camelCase` ( `motorHizi` , `sensorDegeri` )
  - Sınıf ve yapılar için: `PascalCase` ( `MotorKontrolcu` , `RobotKolu` )
  - Sabitler için: `UPPER_SNAKE_CASE` ( `MAX_HIZ` , `PI_SAYISI` )

# Değişken Tanımlama: Geçersiz İsimler

Aşağıdaki tanımlamalar derleme hatası üretir:

- Hatalı: `int 1sensor = 10;` → Değişken ismi rakamla başlayamaz!
- Hatalı: `double motor-hizi;` → Tire `-` eksi operatörü olarak yorumlanır!
- Hatalı: `int sınıf no;` → Değişken adı boşluk içeremez!
- Hatalı: `float class = 5.0;` → C++ anahtar kelimeleri ( `class` , `int` , `return` ) değişken adı olamaz!

Tavsiye: Değişken adlarında Türkçe karakter ('ç', 'ğ', 'ı', 'ö', 'ş', 'ü') kullanmayınız; derleyici uyumluluğunu bozar.

# sizeof Operatörü ve Bellek Boyutları (C++ Kodu)

Her veri tipinin bellekte kapladığı bayt miktarını derleme zamanında döndürür:

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "bool:          " << sizeof(bool)          << " bayt\n";
5      std::cout << "char:         " << sizeof(char)           << " bayt\n";
6      std::cout << "short:        " << sizeof(short)          << " bayt\n";
7      std::cout << "int:          " << sizeof(int)            << " bayt\n";
8      std::cout << "float:        " << sizeof(float)           << " bayt\n";
9      std::cout << "double:       " << sizeof(double)          << " bayt\n";
10     std::cout << "long long:    " << sizeof(long long)       << " bayt\n";
11     return 0;
12 }
```

# Mimariye Göre Veri Tipi Boyutları

Masaüstü 64-bit mimarisindeki tipik konsol çıktısı:

```
1  bool:      1 bayt
2  char:      1 bayt
3  short:     2 bayt
4  int:       4 bayt
5  float:     4 bayt
6  double:    8 bayt
7  long long: 8 bayt
```

- `sizeof` bir fonksiyon çağırısı değil, derleme zamanı (compile-time) operatörüdür.
- Gömülü sistemlerde (8-bit AVR veya 32-bit ARM mikrodenetleyiciler) veri tiplerinin boyutları mimariye göre farklılaşabilir.

# Değişken İlkendirme Biçimleri (Initialization)

C++ dilinde bir değişkene ilk değeri atamanın üç farklı yöntemi bulunur:

```
1 // 1. Kopyalama İlkendirme (C-tarzı)
2 int a = 10;
3
4 // 2. Doğrudan İlkendirme (Direct Initialization)
5 int b(20);
6
7 // 3. Liste İlkendirme (Modern C++11 - Uniform Initialization)
8 int c{30};
9 int d{}; // Varsayılan değeri atanır: 0
```

# Modern C++ Liste İlkendirme (Uniform Initialization)

Neden süslü parantez `{}` ilkendirmesi tercih edilmelidir?

- Daraltıcı Dönüşümü (Narrowing Conversion) Engeller:

```
1 int x = 5.7;    // Derlenir; değer 5'e yuvarlanır, kesirli kısım KAYBOLUR!  
2 int y{5.7};    // DERLEME HATASI: Daraltıcı dönüşüm derleme zamanında engellenir!
```

Akademik Standart: Modern C++ standartlarında değişkenleri süslü parantez `int x{0};` ile ilkendirmek tip güvenliği ve hata önleme açısından en sağlıklı yöntemdir.

# İklendirilmemiş Değişkenler ve Çöp Değer Riski

Bir yerel değişkene ilk değer verilmediğinde bellekte ne oluşur?

```
1  #include <iostream>
2
3  int main() {
4      int sensorDegeri; // İlk değer atanmadı!
5      std::cout << "Deger: " << sensorDegeri << "\n";
6      return 0;
7  }
```

Olası Konsol Çıktısı:

```
1  Deger: 32767      # veya 0, veya -1045239... (Öngörülemez)
```

- Yerel değişkenler yığında (*Stack*) tahsis edilir.
- İlk değer atanmazsa, o bellek alanında önceden kalan rastgele çöp veri (garbage value) okunur.

# Tanımsız Davranış (Undefined Behavior) ve Savunmacı Kodlama

Değişkenleri her zaman başlangıç değeri vererek tanımlayınız:

```
1 int sensorDegeri{0};    // Güvenli: 0 ile başlar
2 double sicaklik{25.0};  // Güvenli: 25.0 ile başlar
3 bool aktifMi{false};    // Güvenli: false ile başlar
```

## Tanımsız Davranış (Undefined Behavior):

İklendirilmemiş bir değişkenin değerini okumak tanımsız davranışa yol açar. Program bazen tesadüfen doğru çalışıyor gibi görünebilir, ancak öngörülemeyen anlarda hatalı hesaplama yapar veya çöker.

# Sabit Değişkenler: `const` Anahtar Kelimesi

Değeri program boyunca değiştirilemez olan değişkenler `const` ile tanımlanır:

```
1  const double PI = 3.1415926535;  
2  const int MAX_MOTOR_HIZI = 255;  
3  const int PULL_UP_PIN = 4;  
4  
5  PI = 3.14; // DERLEME HATASI: assignment of read-only variable 'PI'
```

- `const` değişkenler bildirildikleri anda ilklendirilmek zorundadır:

```
1  const int esik; // DERLEME HATASI: uninitialized const 'esik'
```

# const Kullanımının Tasarımsal ve Donanımsal Faydaları

## 1. Hataların Derleme Zamanında Yakalanması:

- Yanlışlıkla kritik bir donanım limitinin değiştirilmesini derleyici önler.

## 2. Kod Okunabilirliği ve Niyet Bildirimi:

- Kodu inceleyen mühendis ilgili büyüklüğün sistem sabiti olduğunu net biçimde anlar.

## 3. Derleyici Optimizasyonu:

- Derleyici sabit değeri doğrudan makine talimatlarına gömebilir ve bellek erişim maliyetini düşürür.

# Otomatik Tip Çıkarımı: `auto` (C++11)

Modern C++11 ile derleyici, atanan başlangıç değerinden türü otomatik çıkarabilir:

```
1 auto x = 10;           // int olarak belirlenir
2 auto y = 3.14;         // double olarak belirlenir
3 auto z = 'A';           // char olarak belirlenir
4 auto aktif = true;      // bool olarak belirlenir
```

- `auto` dinamik bir tip (Python türü) değildir.
- Tür, derleme zamanında kesinleştirilir ve çalışma zamanında kesinlikle değiştirilemez:

```
1 auto sayi = 5; // int türü belirlendi
2 sayi = "Metin"; // DERLEME HATASI!
```

# auto Kullanım Kuralları ve Tercih Kriterleri

- Başlangıç değeri zorunludur:

```
1 auto gecersiz; // DERLEME HATASI: Tip çıkarımı için başlangıç değeri şarttır!
```

- Tercih Kriterleri:

- İlerleyen haftalarda ele alacağımız karmaşık STL iteratör tiplerinde kodu sadeleştirmek için kullanılır:

```
1 // Uzun ve karmaşık tür bildirimi yerine:  
2 std::vector<int>::const_iterator it = vec.cbegin();  
3 // auto ile sade ve açık sözdizimi:  
4 auto it = vec.cbegin();
```

- Basit değişkenlerde ( `int sayi{10};` ) türün açıkça yazılması kodun doğrudan kavranması açısından tercih edilir.

# Aritmetik Operatörler

C++ dilindeki temel matematiksel işlem operatörleri:

| Operatör | Anlamı        | Örnek | Sonuç ( a=10, b=3 )  |
|----------|---------------|-------|----------------------|
| +        | Toplama       | a + b | 13                   |
| -        | Çıkarma       | a - b | 7                    |
| *        | Çarpma        | a * b | 30                   |
| /        | Bölme         | a / b | 3 (tamsayı bölmesi!) |
| %        | Modül (Kalan) | a % b | 1                    |

Modül Operatörü Notu: % operatörü yalnızca tamsayılar arasında çalışır; ondalıklı sayılarda ( float/double ) kullanılamaz.

# Operatörlerde İşlem Önceliği

C++ operatörleri belirli bir öncelik sırasına göre işletilir:

1. Parantezler: `()` → En yüksek öncelik
2. Birli (Unary) Operatörler: `++`, `--`, `+`, `-`, `!`
3. Çarpma, Bölme, Modül: `*`, `/`, `%`
4. Toplama ve Çıkarma: `+`, `-`
5. İlişkisel Operatörler: `<`, `<=`, `>`, `>=`
6. Eşitlik Operatörleri: `==`, `!=`
7. Mantıksal VE / VEYA: `&&`, `||`
8. Atama Operatörleri: `=`, `+=`, `--`, `*=`, `/=` → En düşük öncelik

**Tavsiye:** Karışıklığı önlemek için karmaşık ifadelerde her zaman açıkça parantez `()` kullanınız.

# Tam Sayı Bölmesi Tuzağı ve Veri Kaybı Riski

C++ dilinde iki tamsayı bölündüğünde sonuç daima bir tamsayıdır (kesirli kısım atılır):

```
1  int a = 5;  
2  int b = 2;  
3  double sonuc = a / b; // Sonuç: 2.0 (2.5 DEĞİL!)
```

- `a / b` aritmetik işlemi yürütüldüğünde ara sonuç `2` tamsayısı olarak üretilir.
- Ardından bu tamsayı `double` türüne dönüştürülerek `2.0` olarak atanır; kesir verisi geri dönülmez biçimde kaybolmuştur.

# Güvenli Tür Dönüşümü: `static_cast`

Hassas ondalık sonuç elde etmek için işlenenlerden en az biri ondalıklı tipe dönüştürülmelidir:

```
1  int a = 5;
2  int b = 2;
3
4  // Modern C++ standardı: static_cast
5  double sonuc = static_cast<double>(a) / b; // Sonuç: 2.5
```

- `static_cast<HedefTip>(ifade)` :
  - Derleme zamanında tür dönüşümünün mantıksal geçerliliğini denetler.
  - C-tarzı `(double)a` kalıbına kıyasla tür güvenliği sağlar ve kod içinde kolayca taranabilir.

# İlişkisel (Karşılaştırma) Operatörleri

İki değeri karşılaştırır ve `bool` türünde sonuç üretir ( `true` / `false` ):

| Operatör           | Açıklama            | Örnek                  | Sonuç ( <code>x = 5</code> ) |
|--------------------|---------------------|------------------------|------------------------------|
| <code>==</code>    | Eşit mi?            | <code>x == 5</code>    | <code>true</code>            |
| <code>!=</code>    | Eşit değil mi?      | <code>x != 5</code>    | <code>false</code>           |
| <code>&lt;</code>  | Küçük mü?           | <code>x &lt; 10</code> | <code>true</code>            |
| <code>&gt;</code>  | Büyük mü?           | <code>x &gt; 10</code> | <code>false</code>           |
| <code>&lt;=</code> | Küçük veya eşit mi? | <code>x &lt;= 5</code> | <code>true</code>            |
| <code>&gt;=</code> | Büyük veya eşit mi? | <code>x &gt;= 5</code> | <code>true</code>            |

Kritik Hata: Tek eşittir `=` değer atar; çift eşittir `==` karşılaştırma yapar. Koşul içinde `if (x = 5)` yazmak mantık hatasına yol açar.

# Mantıksal Operatörler ve Kısa Devre Değerlendirmesi

Birden fazla mantıksal koşulu birleştirmek için kullanılır:

| Operatör | Adı                   | Açıklama                                                             |
|----------|-----------------------|----------------------------------------------------------------------|
| &&       | Mantıksal VE (AND)    | Her iki koşul da <code>true</code> ise sonuç <code>true</code>       |
|          | Mantıksal VEYA (OR)   | Koşullardan en az biri <code>true</code> ise sonuç <code>true</code> |
| !        | Mantıksal DEĞİL (NOT) | Koşulun tersini alır ( <code>!true == false</code> )                 |

Kısa Devre (Short-Circuit) Değerlendirmesi:

`A && B` ifadesinde `A` yanlışsa, `B` hiç çalıştırılmaz.

`A || B` ifadesinde `A` doğruysa, `B` hiç çalıştırılmaz.

# Bileşik Atama Operatörleri

Aritmetik işlem ve atamayı tek bir adımda birleştiren operatörler:

| Operatör        | Örnek                | Açılımı                 |
|-----------------|----------------------|-------------------------|
| <code>+=</code> | <code>x += 5;</code> | <code>x = x + 5;</code> |
| <code>-=</code> | <code>x -= 3;</code> | <code>x = x - 3;</code> |
| <code>*=</code> | <code>x *= 2;</code> | <code>x = x * 2;</code> |
| <code>/=</code> | <code>x /= 4;</code> | <code>x = x / 4;</code> |
| <code>%=</code> | <code>x %= 3;</code> | <code>x = x % 3;</code> |

# Artırma ve Azaltma Operatörleri: Önek vs Sonek

```
1  int a = 5;  
2  int b = ++a; // Önek (Prefix): Önce a artırılır (6), ardından b'ye atanır (b = 6)  
3  
4  int x = 5;  
5  int y = x++; // Sonek (Postfix): Önce x'in mevcut değeri atanır (y = 5), sonra x artar (x
```

Performans Notu: Temel tamsayılarda döngü içinde `++i` ve `i++` aynı optimizasyona uğrar; ancak nesne yönelimli sınıflarda (örneğin STL iteratörlerinde) önek `++i` geçici kopya nesne üretmediği için daha verimlidir.

# İsim Alanı (Namespace) Mimarisi ve Çakışma Önleme

Farklı kütüphanelerin veya modüllerin aynı isimdeki değişken ya da fonksiyonları tanımlaması durumunda oluşacak isim çakışmalarını engeller:

```
1 namespace MotorKutuphanesi {  
2     int maxHiz = 255;  
3     void baslat() { /* Donanım başlatma */ }  
4 }  
5  
6 namespace SensorKutuphanesi {  
7     int maxHiz = 100; // Çakışma oluşmaz; farklı isim alanı  
8     void baslat() { /* Sensör başlatma */ }  
9 }
```

# Kapsam Çözümleme Operatörü ( :: ) ile Erişim

İsim alanı içinde tanımlanmış bir elemana erişmek için :: (Scope Resolution Operator) kullanılır:

```
1  int main() {  
2      // Kapsam operatörü ile doğrudan erişim:  
3      MotorKutuphanesi::baslat();  
4      int motorLimit = MotorKutuphanesi::maxHiz;  
5  
6      SensorKutuphanesi::baslat();  
7      int sensLimit = SensorKutuphanesi::maxHiz;  
8  
9      return 0;  
10 }
```

# using namespace std; Direktifinin Riskleri

Başlangıç seviyesi kaynaklarda sıklıkla görülen bu kullanım:

```
1  #include <iostream>
2  using namespace std; // TÜM std isim alanını global kapsama açar!
3
4  int main() {
5      cout << "Merhaba\n";
6  }
```

Neden profesyonel C++ mühendisliğinde ve bu derste önerilmez?

- `std` isim alanı içerisinde binlerce standart fonksiyon ve tür mevcuttur ( `count` , `max` , `min` , `move` , `swap` vb.).
- Tanımladığınız bir değişken veya fonksiyon standart isimlerle çakışır.

# İsim Çakışması Senaryosu ve Derleyici Belirsizlik Hatası

```
1  #include <iostream>
2  using namespace std;
3
4  // Projeye ait global değişken:
5  int count = 10;
6
7  int main() {
8      // DERLEME HATASI: std::count fonksiyonu ile global count çakışır!
9      cout << count << endl;
10     return 0;
11 }
```

Derleyici Çıktısı:

```
1  error: reference to 'count' is ambiguous
2  note: candidates are: 'int count'
3  note: and 'template<class ...> count'
```

# Güvenli İsim Alanı Kullanım Stratejileri

## 1. Açık Kapsam Bildirimi (En Güvenli ve Önerilen Standart):

```
1 std::cout << "Marmara Mekatronik\n";  
2 std::cin >> olculenDeger;
```

## 2. Yalnızca İhtiyaç Duyulan Sembolleri Dahil Etme:

```
1 using std::cout;  
2 using std::endl;  
3  
4 cout << "Sistem Hazır" << endl; // Yalnızca cout ve endl açılmıştır
```

# İsim Alanı İlkeleri ve Akademik Standart

## Mühendislik Kuralı:

Başlık dosyalarında ( `.h` / `.hpp` ) **KESİNLİKLE** `using namespace` kullanılmaz! İlgili başlığı dahil eden tüm kaynak dosyaların global alanını kirletir.

Ders Standardı: Bu ders kapsamındaki tüm sunumlarda, laboratuvarlarda ve sınav teslimlerinde `std::cout` , `std::cin` , `std::endl` şeklinde açık kapsam bildirme kuralı uygulanır.

# Standart Çıktı Akışı: `std::cout` ve `<<`

`std::cout` karakter tabanlı çıktı akışıdır:

```
1  #include <iostream>
2
3  int main() {
4      int motorHizi = 1500;
5      double gerilim = 12.6;
6
7      // Zincirleme akış yönlendirme:
8      std::cout << "Motor Hızı: " << motorHizi << " RPM, "
9                << "Gerilim: "    << gerilim    << " V\n";
10
11     return 0;
12 }
```

- `<<` (*Stream Insertion Operator*): Sağına yazılan değişkeni veya metni çıktı akışına yönlendirir.
- `\n` vs `std::endl` : `\n` yalnızca yeni satır ekler; `std::endl` satır ekleyip arabelleği anında temizler (*flush*).

# Standart Girdi Akışı: `std::cin` ve `>>`

Klavyeden veri okumak için kullanılır:

```
1  #include <iostream>
2
3  int main() {
4      int hedefHiz;
5      std::cout << "Hedef motor hızınızı giriniz (RPM): ";
6      std::cin >> hedefHiz;
7
8      std::cout << "Ayarlanan Hız: " << hedefHiz << " RPM\n";
9      return 0;
10 }
```

- `>>` (*Stream Extraction Operator*): Akıştan gelen veriyi çözer ve sağındaki değişkene kaydeder.
- Boşluk (*whitespace*) karakterlerine ( `' '`, `'\t'`, `'\n'` ) kadar olan kısmı okur.

# Standart Girdi Akışında ( `cin` ) Hata Yönetimi

Kullanıcı tamsayı beklenen bir akışa geçersiz karakter (harf/sembol) girdiğinde akış kilitlenir:

```
1  #include <iostream>
2
3  int main() {
4      int sayi;
5      std::cout << "Bir tam sayı giriniz: ";
6      std::cin >> sayi;
7
8      if (std::cin.fail()) {
9          std::cout << "HATA: Geçersiz tamsayı girdisi algılandı!\n";
10     } else {
11         std::cout << "Girilen sayı: " << sayi << "\n";
12     }
13     return 0;
14 }
```

# Girdi Arabelleği Temizleme ( `cin.clear` ve `cin.ignore` )

Hatalı girdiden sonra programın yeni veri okuyabilmesi için iki işlem zorunludur:

```
1  if (std::cin.fail()) {
2      // 1. Akışın hata bayrağını temizle
3      std::cin.clear();
4
5      // 2. Arabellekte biriken hatalı karakterleri satır sonuna kadar temizle
6      std::cin.ignore(10000, '\n');
7
8      std::cout << "Lütfen geçerli bir sayı giriniz.\n";
9  }
```

- `std::cin.clear()` : Akış durum bayrağını sıfırlayarak `good` durumuna getirir.
- `std::cin.ignore(n, '\n')` : Arabellekte kalan karakterleri `\n` görene kadar atar.

# <iomanip> Kütüphanesi ve Formatlı Çıktı (C++ Kodu)

Telemetri tabloları ve hassas mühendislik çıktıları için formatlama:

```
1  #include <iostream>
2  #include <iomanip> // Format manipulatorleri
3
4  int main() {
5      double pi = 3.1415926535;
6
7      // Sabit ondalıklı ve 2 basamak hassasiyet:
8      std::cout << std::fixed << std::setprecision(2);
9      std::cout << "Pi (2 basamak): " << pi << "\n"; // 3.14
10
11     std::cout << std::setprecision(4);
12     std::cout << "Pi (4 basamak): " << pi << "\n"; // 3.1416
13
14     // Tablo Formatı ve Sütun Genişliği:
15     std::cout << std::setw(10) << "Ürün" << std::setw(8) << "Fiyat" << "\n";
16     std::cout << std::setw(10) << "Motor" << std::setw(8) << 250.50 << "\n";
17     return 0;
18 }
```

# Standart Akış Manipülatörleri ve Görevleri

- `std::fixed` :
  - Ondalık gösterimi sabitler; bilimsel ( `3.14e+00` ) gösterim yerine standart gösterim sağlar.
- `std::setprecision(n)` :
  - Virgülden sonra gösterilecek basamak sayısını belirler (otomatik yuvarlama yapar).
- `std::setw(n)` :
  - Bir sonraki çıktının alan genişliğini `n` karakter olarak ayarlar (sağa dayalı tablo basımında kullanılır).
- `std::left` / `std::right` :
  - Verinin sütun içerisindeki hizalama yönünü belirler.

# Hata Akışı: `std::cerr` ile Arabelleksiz Çıktı

Kritik arıza ve hata durumlarında standart hata akışı kullanılır:

```
1  #include <iostream>
2
3  int main() {
4      int pay = 10, payda = 0;
5
6      if (payda == 0) {
7          // Standart hata akışı (Arabelleksiz - Unbuffered)
8          std::cerr << "[HATA]: Sıfıra bölme yapılamaz!" << std::endl;
9          return 1; // Hata durum kodu ile sonlandırma
10     }
11
12     std::cout << "Sonuç: " << pay / payda << "\n";
13     return 0;
14 }
```

# `std::cout` (Buffered) ile `std::cerr` (Unbuffered) Karşılaştırması

- `std::cout` (Arabellekli - Buffered):
  - Normal çalışma çıktıları için kullanılır.
  - Veriler sistem arabelleğinde toplanır ve verimli paketler halinde iletilir; dosyaya yönlendirilebilir ( `./prog > cikti.txt` ).
- `std::cerr` (Arabelleksiz - Unbuffered):
  - Hata, uyarı ve arıza telemetrisi için kullanılır.
  - Arabellekte bekletilmez; program çökse dahi anında ekrana basılır.
  - Standart çıktıdan ayrı log dosyasına yönlendirilebilir ( `./prog 2> hata.log` ).

# Alıştırma 1: Dört İşlem (Problem Tanımı)

Kullanıcıdan iki tam sayı alarak toplam, fark, çarpım ve bölüm sonuçlarını hesaplayan programı tasarlayınız. Sıfıra bölme durumuna karşı savunmacı kontrol uygulayınız.

Beklenen Konsol Çıktısı:

```
1 Birinci sayıyı girin: 10
2 İkinci sayıyı girin: 4
3 Toplam: 14
4 Fark: 6
5 Çarpım: 40
6 Bölüm: 2.5
```

# Alıştırma 1: C++ Çözümü

```
1  #include <iostream>
2
3  int main() {
4      int sayi1, sayi2;
5      std::cout << "Birinci sayıyı girin: ";
6      std::cin >> sayi1;
7      std::cout << "İkinci sayıyı girin: ";
8      std::cin >> sayi2;
9
10     int toplam = sayi1 + sayi2;
11     int fark = sayi1 - sayi2;
12     int carpim = sayi1 * sayi2;
13
14     std::cout << "Toplam: " << toplam << "\n";
15     std::cout << "Fark: " << fark << "\n";
16     std::cout << "Çarpım: " << carpim << "\n";
17
18     if (sayi2 != 0) {
19         double bolum = static_cast<double>(sayi1) / sayi2;
```

## Alıştırma 2: Daire Alanı ve Çevresi (Problem Tanımı)

Kullanıcıdan yarıçap değeri alarak dairenin alanını hesaplayınız:

$$\text{Alan} = \pi \cdot r^2$$

$\pi = 3.14159$  sabit ( `const` ) olarak tanımlanmalı ve sonuç 2 ondalık basamak hassasiyetle yazdırılmalıdır.

Beklenen Konsol Çıktısı:

```
1 Yarıçapı girin: 7
2 Dairenin alanı: 153.94
```

## Alıştırma 2: C++ Çözümü

```
1  #include <iostream>
2  #include <iomanip>
3
4  int main() {
5      const double PI = 3.14159;
6      double yaricap;
7
8      std::cout << "Yarıçapı girin: ";
9      std::cin >> yaricap;
10
11     double alan = PI * yaricap * yaricap;
12
13     std::cout << std::fixed << std::setprecision(2);
14     std::cout << "Dairenin alanı: " << alan << "\n";
15
16     return 0;
17 }
```

# Alıştırma 3: Karakter ve ASCII Karşılığı (Problem Tanımı)

Kullanıcıdan tek bir karakter alarak bu karakterin bellekteki sayısal ASCII kodunu yazdıran programı tasarlayınız. Tür dönüşümünde `static_cast` kullanılmalıdır.

Beklenen Konsol Çıktısı:

```
1  Bir karakter girin: A
2  Karakter: A
3  ASCII Karşılığı: 65
```

## Alıştırma 3: C++ Çözümü

```
1  #include <iostream>
2
3  int main() {
4      char karakter;
5      std::cout << "Bir karakter girin: ";
6      std::cin >> karakter;
7
8      // char türünden int türüne güvenli dönüşüm
9      int asciiKodu = static_cast<int>(karakter);
10
11     std::cout << "Karakter: " << karakter << "\n";
12     std::cout << "ASCII Karşılığı: " << asciiKodu << "\n";
13
14     return 0;
15 }
```

## Alıştırma 4: Sayı İşareti Kontrolü (Problem Tanımı)

Girilen bir tam sayının pozitif, negatif ya da sıfır olduğunu kontrol eden programı tasarlayınız (Hafta 2 karar yapıları önizlemesi).

Beklenen Konsol Çıktısı:

```
1 Bir sayı girin: -7
2 Sayı negatiftir.
```